



Programming Exercise 00

Expression Evaluation

Developers

- *Royo, Dave Christian*
- *Siosana, Cedric*
- *Tabon, Jeycel Ann*

Tools used

- *Python*
- *Tkinter*

Program Description

The program is a code processing application with a graphical user interface (GUI) that allows users to enter or load input code and process each line according to the given program specification. It accepts assignment statements and arithmetic expressions, converts expressions from infix notation to postfix notation, evaluates them, and manages the values of variables throughout the input. The program also detects errors such as invalid input, undefined variables, and division by zero. The GUI provides an editable input text area, a non-editable output text area, a Load File button for opening .in files from any directory, and a Process button for processing the entered or loaded code. After processing, the program displays the postfix notation, evaluated results, final variable values, and any errors encountered in the required output format.

Functions

Since the program was developed by a group of three members, we divided the system into separate modules based on each member's assigned responsibility. Each member developed and maintained their own module, such as the GUI and file-handling module, expression-processing module, and code parsing, variable management, and output module. The modules were designed to work together by calling the functions of the other modules whenever their functionality was needed. This modular approach allowed each member to work independently on their assigned part while still enabling the modules to communicate and function as one complete program. After developing the individual modules, the group



integrated and tested them together to ensure that the complete system followed the required program specification.

Module 1: GUI and File Handling

ExpressionEvaluatorApp – Manages the graphical user interface, including the input and output text areas, file loading, and program controls.

- `__init__(self, root)` – Initializes the application window and sets up the GUI components.
- `_build_ui(self)` – Creates and arranges the input and output text areas, labels, and buttons.
- `on_load_file(self)` – Opens a .in file, reads its contents, and displays them in the input area, replacing any existing input.
- `on_process(self)` – Validates the input, sends it to the processing module, and displays the generated output in the output area.

Module 2: Expression Conversion and Evaluation

- `get_tokens(expression)` – Tokenizes the expression and identifies numbers, variables, operators, and parentheses.
- `get_precedence(token)` – Determines operator precedence.
- `is_left_associative(token)` – Determines operator associativity.
- `infix_to_postfix(tokens)` – Converts infix expressions into postfix notation using the Shunting-yard algorithm.
- `postfix_to_string(postfix_tokens)` – Formats postfix tokens for display.
- `format_number(value)` – Formats evaluated numeric results.
- `evaluate_postfix(postfix_tokens, variables)` – Evaluates postfix expressions using the provided variable values and handles arithmetic and evaluation errors.



Module 3: Code Parsing, Variables, Errors, and Output

This module is responsible for processing complete input lines, managing variables, handling program errors, and generating the required output.

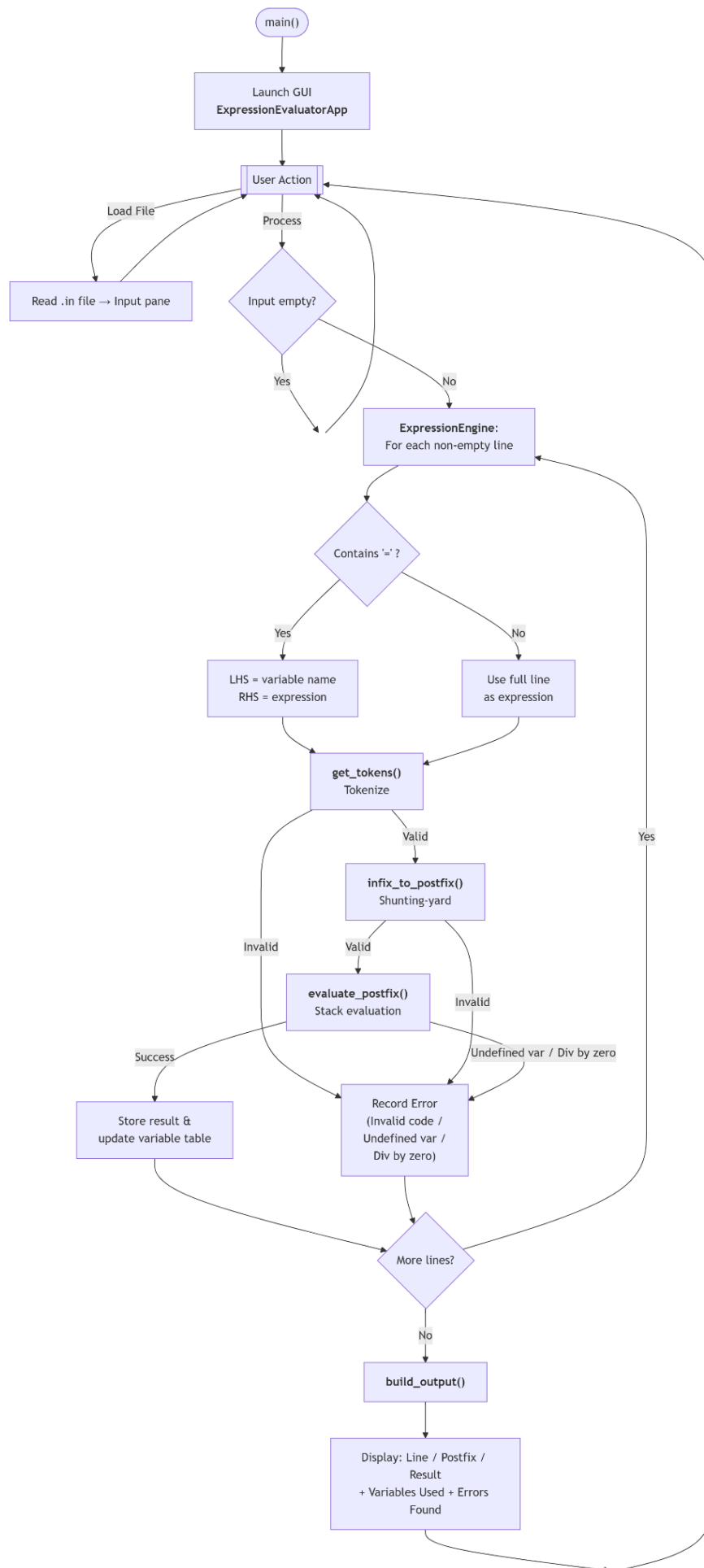
- `is_valid_variable_name(name)` – Validates variable names according to the required naming rules.
- `LineRecord` – Stores the information for each processed line, including the original line, postfix expression, and result.
- `ExpressionEngine.__init__()` – Initializes the variable storage, processed line records, and error records.
- `process_all(input_text)` – Processes all input lines and coordinates the processing of each line.
- `_process_single_line(original_line, line)` – Parses each input line, determines whether it is an assignment or expression, validates the code, calls the expression-processing functions, manages variable assignments, and handles errors.
- `build_output()` – Generates the final formatted output containing processed lines, variable values, and errors.
- `format_number(value)` – Formats numerical results for display in the required output format.



UNIVERSITY OF THE PHILIPPINES MINDANAO
Brgy. Mintal, Tugbok District, Davao City 8022 Davao del Sur
College of Science and Mathematics | E: esm.upmindanao@up.edu.ph



Control Flow





Work Distribution

Royo:

He was responsible for the code interpretation and management of variables. This included parsing each input line to determine whether it was an assignment statement or an expression, validating assignment formats and variable naming rules, and identifying invalid inputs. Royo also managed the storage and retrieval of variable values, including keeping the most recently assigned value and detecting the use of undefined variables. In addition, he handled errors such as invalid assignments, undefined variables, and errors that occurred during evaluation, including preserving a variable's previous value when an assignment failed. He was also responsible for displaying the required final output, including the original input line, postfix notation, evaluated result, variables and their final values, detected errors, and proper blank-line separation.

Siosana:

Siosana was responsible for developing the main graphical user interface and handling user interaction with the program. This included creating the input text area, non-editable output text area, Load File button, and Process button. Siosana implemented the file-loading functionality, allowing users to select and open .in files from any directory, read their contents, and display them in the input area while replacing any previously loaded input. Siosana integrated the GUI with the processing modules developed by Royo and Tabon so that the complete program could receive input, process it, and display the required output. Finally, Siosana also created the executable file for testing and evaluation,

Tabon:

Tabon was responsible for developing the core expression-processing engine of the program. This included processing the expression portion of each input line, converting infix expressions into postfix notation, and evaluating the resulting postfix expressions. The



UNIVERSITY OF THE PHILIPPINES MINDANAO
Brgy. Mintal, Tugbok District, Davao City 8022 Davao del Sur
College of Science and Mathematics | E: csn.upmindanao@up.edu.ph



module supported the required arithmetic operators, including addition (+), subtraction (-), multiplication (*), division (/), and modulo (%). Tabon handled both simple expressions, such as a single value or variable, and complex expressions involving multiple operators. The expression engine returned the postfix expression and evaluated the result while also detecting evaluation-related errors, particularly division-by-zero errors.